

EVALUASI TENGAH SEMESTER (ETS) ALGORITMA DAN PEMROGRAMAN

Semester Genap 2025/2026

PROJECT BASED LEARNING (PBL) Pengembangan Sistem Instrumentasi Pengukuran dan Kontrol Berbasis Rust

**Studi Kasus: Sistem Monitoring Kelembaban dan Kualitas
Udara**

Disusun Oleh:

Hageng Priyambada Alim	(NRP: 2042251089)
Aurepope Kasih Lamarebon	(NRP: 2042241122)

Dosen Pengampu:
Ahmad Radhy, S.Si., M.Si.

**DEPARTEMEN TEKNIK INSTRUMENTASI
INSTITUT TEKNOLOGI SEPULUH NOPEMBER
SURABAYA
2026**

Contents

1	Pendahuluan	2
1.1	Kajian Pustaka dan Sitasi	2
2	Studi Kasus Instrumentasi	2
3	Algoritma dan Flowchart	3
3.1	Algoritma Sistem	3
3.2	Flowchart	4
4	Penjelasan Program Rust	5
5	Konsep Pemrograman Berbasis Objek (OOP)	5
6	Implementasi Komputasi Numerik	5
7	Hasil Pengujian	5
7.1	Skenario 1: Kondisi Lingkungan Normal (Alarm Non-Aktif)	8
7.2	Skenario 2: Peringatan Ambang Batas (Alarm Aktif)	9
8	Analisis Hasil	9
9	Kesimpulan	9

1 Pendahuluan

Sistem pemantauan kualitas udara dan kondisi lingkungan kini menjadi kebutuhan krusial dalam perlindungan kesehatan publik dan lingkungan. Penggunaan teknologi *Internet of Things* (IoT) memungkinkan perolehan informasi kualitas udara secara *real-time* dan akurat.

Proyek ETS ini bertujuan mengembangkan purwarupa perangkat lunak berbasis Rust untuk memantau kelembaban dan konsentrasi gas polutan. Rust dipilih karena keunggulannya dalam keamanan memori, yang sangat penting untuk aplikasi instrumentasi yang berjalan terus-menerus.

1.1 Kajian Pustaka dan Sitasi

Sistem ini dirancang dengan merujuk pada integrasi berbagai sensor seperti DHT11 untuk suhu dan kelembaban, serta MQ135 untuk menganalisis kadar gas di lingkungan. Sebagaimana dijelaskan oleh Garg dkk. (2025), sensor DHT11 mendeteksi suhu dan kelembaban melalui *output* digital yang dihubungkan langsung ke mikrokontroler seperti ESP32.

Selain aspek pembacaan, sistem keamanan otomatis menjadi komponen vital. Chanthakit dan Rattanapoka (2018) menekankan bahwa ketika nilai data sensor melebihi rentang yang dikonfigurasi, sistem harus mampu mengirimkan pesan alarm atau notifikasi untuk memperingatkan pengguna secara cepat. Logika ambang batas (*set point*) inilah yang diimplementasikan dalam pengembangan objek kontroler pada proyek ini.

2 Studi Kasus Instrumentasi

Studi kasus yang diangkat adalah **Sistem Monitoring Ruangan Industri**. Sistem melakukan akuisisi data dari dua sensor virtual:

- **Sensor DHT11:** Membaca suhu ($^{\circ}\text{C}$) dan kelembaban (%). Sistem menyimulasikan adanya *offset* pembacaan sebesar +1.0 untuk suhu dan +2.0 untuk kelembaban.
- **Sensor MQ135:** Menghitung konsentrasi gas CO_2 dan NH_3 (ppm) berdasarkan korelasi linear dengan tingkat kelembaban udara.

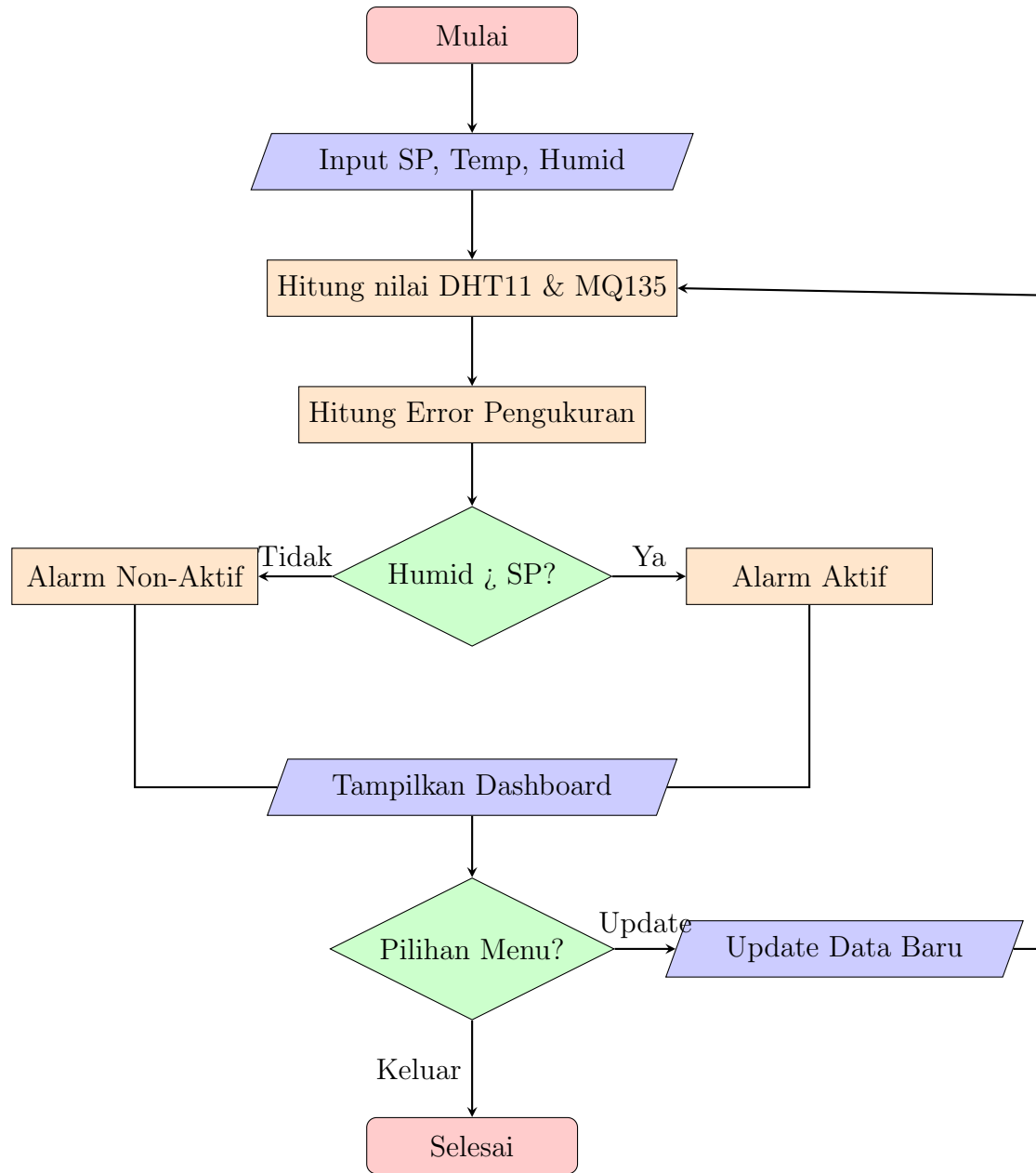
Sistem menyimulasikan *Controller ESP32* yang bertugas mengaktifkan aktuator Alarm jika kelembaban aktual atau terbaca melampaui batas aman (*Set Point*) yang ditentukan operator.

3 Algoritma dan Flowchart

3.1 Algoritma Sistem

1. Operator menginput *Set Point* kelembaban dan nilai kondisi aktual ruangan.
2. Program menghitung nilai sensor DHT11 dengan menambahkan galat statis.
3. Program mengekstrapolasi nilai gas MQ135 dari data kelembaban.
4. Dilakukan komputasi numerik untuk menghitung *error* pembacaan.
5. Jika nilai kelembaban $>$ *Set Point*, aktuator Alarm berubah status menjadi "Aktif".
6. Dashboard ditampilkan ke terminal dan operator dapat memilih menu pembaruan data atau keluar.

3.2 Flowchart



4 Penjelasan Program Rust

Program menggunakan *library* I/O standar Rust (`std::io`). Fungsi modular `input_angka` menjamin validitas data melalui *error handling* menggunakan `match` atau `if let` pada hasil *parsing* string ke float. Struktur program menggunakan loop tak terbatas yang hanya bisa dihentikan jika pengguna memilih menu keluar (`break`).

5 Konsep Pemrograman Berbasis Objek (OOP)

Pemrograman berbasis objek pada aplikasi ini diimplementasikan melalui enkapsulasi data dalam `struct Ruangan`. Logika sensor dan kontroler dipisahkan dalam blok `impl Ruangan`, yang memungkinkan penggunaan metode instansi (`&self`) untuk mengakses atribut objek secara aman.

6 Implementasi Komputasi Numerik

Sistem menerapkan metode numerik sederhana untuk analisis ketidakpastian instrumentasi melalui perhitungan galat (*error*) absolut:

$$Error = Nilai_{Terbaca} - Nilai_{Aktual} \quad (1)$$

Hal ini diterapkan pada fungsi `error_temp()` dan `error_humid()` guna memantau akurasi sensor virtual terhadap kondisi *ground truth*.

7 Hasil Pengujian

Pengujian sistem dilakukan dengan menjalankan program simulasi kontrol berbasis teks dan memantau antarmuka *dashboard* pada terminal. Berikut adalah *source code* bahasa Rust yang digunakan sebagai pengendali utama dalam sistem instrumentasi ini:

```
1 use std::io;
2
3 // =====
4 //  Struct Ruangan      hanya 3 field input utama
5 // =====
6 struct Ruangan {
7     set_point_lembab: f32,
8     temp_aktual: f32,
9     humid_aktual: f32,
10 }
11
12 // =====
13 //  Implementasi Method
14 // =====
15 impl Ruangan {
16     // Inisialisasi dengan 3 input
17     fn new(set_point_lembab: f32, temp_aktual: f32, humid_aktual:
        f32) -> Ruangan {
```

```

18     Ruangan {
19         set_point_lembab,
20         temp_aktual,
21         humid_aktual,
22     }
23 }
24
25 // DHT11: pembacaan sensor = aktual + offset kecil (1 atau 2)
26 fn dht11_temp(&self) -> f32 {
27     self.temp_aktual + 1.0
28 }
29
30 fn dht11_humid(&self) -> f32 {
31     self.humid_aktual + 2.0
32 }
33
34 // MQ135: CO2 dan NH3 dihitung dari pembacaan humidity DHT11
35 fn mq135_co2(&self) -> f32 {
36     self.dht11_humid() * 5.0
37 }
38
39 fn mq135_nh3(&self) -> f32 {
40     self.dht11_humid() * 10.0
41 }
42
43 // Cek apakah aktuator alarm harus aktif
44 fn aktuator_aktif(&self) -> bool {
45     self.humid_aktual > self.set_point_lembab ||
46     self.dht11_humid() > self.set_point_lembab
47 }
48
49 // Error = pembacaan sensor - aktual
50 fn error_temp(&self) -> f32 {
51     self.dht11_temp() - self.temp_aktual
52 }
53
54 fn error_humid(&self) -> f32 {
55     self.dht11_humid() - self.humid_aktual
56 }
57
58 // Tampilkan seluruh output
59 fn tampilkan(&self) {
60     let status_alarm = if self.aktuator_aktif() { "Aktif" }
61         else { "Non-Aktif" };
62     println!("\n=====");
63     println!("          DASHBOARD MONITORING RUANGAN          ");
64     println!("=====");
65
66     // --- DHT11 ---
67     println!("\n[ Pembacaan Sensor DHT11 ]");
68     println!("  Temperature  : {:.1} C ", self.dht11_temp());
69     println!("  Humidity      : {:.1} %", self.dht11_humid());
70
71     // --- MQ135 ---
72     println!("\n[ Pembacaan Sensor MQ135 ]");
73     println!("  CO2 : {:.1} ppm", self.mq135_co2());

```

```

73     println!("  NH3 : {:.1} ppm", self.mq135_nh3());
74
75     // --- Controller ESP32 ---
76     println!("\n[ Controller ESP32 ]");
77     println!("  Sensor (DHT11 dan MQ135) : Aktif");
78     println!("  Aktuator (Alarm)           : {}", status_alarm);
79
80     // --- Alarm ---
81     println!("\n[ Alarm ]");
82     if self.aktuator_aktif() {
83         println!("  Status : AKTIF  !! Humidity melebihi Set
84           Point !!");
85     } else {
86         println!("  Status : Non-Aktif");
87     }
88
89     // --- Error ---
90     println!("\n[ Error Sensor ]");
91     println!("  Error Temperature : {:.1} C ", self.error_temp
92       ());
93     println!("  Error Humidity      : {:.1} %", self.error_humid
94       ());
95
96     println!("\n=====\\n");
97 }
98
99 // =====
100 //  Helper input
101 // =====
102 fn input_angka(pesan: &str) -> f32 {
103     loop {
104         println!("{}", pesan);
105         let mut input = String::new();
106         io::stdin().read_line(&mut input).unwrap();
107         if let Ok(angka) = input.trim().parse() {
108             return angka;
109         } else {
110             println!("Input salah! Masukkan angka.");
111         }
112     }
113 }
114
115 // =====
116 //  Main
117 // =====
118 fn main() {
119     println!("=====");
120     println!("      SISTEM MONITORING SENSOR v3.0      ");
121     println!("=====");
122     let sp_lembab      = input_angka("\nMasukkan Set Point Humidity
123       (%): ");
124     let temp_aktual    = input_angka("Masukkan Temperature Aktual (
125       C ): ");
126     let humid_aktual   = input_angka("Masukkan Humidity Aktual (%): "
127       );

```

```

123
124     let mut ruangan = Ruangan::new(sp_lembab, temp_aktual,
125         humid_aktual);
126     ruangan.tampilkan();
127
128     // Loop menu update
129     loop {
130         println!("MENU: [1] Update Nilai [2] Keluar");
131         let pilihan = input_angka("Pilih (1/2): ");
132
133         if pilihan == 1.0 {
134             let t = input_angka("\nTemperature Aktual baru ( C ): ");
135             let h = input_angka("Humidity Aktual baru (%): ");
136             ruangan.temp_aktual = t;
137             ruangan.humid_aktual = h;
138             ruangan.tampilkan();
139         } else if pilihan == 2.0 {
140             println!("Sistem Dimatikan.");
141             break;
142         } else {
143             println!("Pilihan tidak valid.");
144         }
145     }

```

Berdasarkan implementasi kode di atas, pengujian dilakukan dalam dua skenario dinamis untuk melihat respons logika kontrol aktuator terhadap perubahan nilai sensor di lingkungan virtual.

7.1 Skenario 1: Kondisi Lingkungan Normal (Alarm Non-Aktif)

Pada pengujian pertama, sistem membaca data dari sensor dengan hasil suhu lingkungan 51.0°C dan kelembaban 42.0% . Karena nilai kelembaban terbaca masih berada di bawah atau setara dengan batas *Set Point* yang ditetapkan oleh operator, metode `aktuator_aktif()` bernilai *false*. Oleh karena itu, Controller ESP32 menahan aktuator dalam status **Non-Aktif**.

Sistem juga menunjukkan akurasi kalkulasi error sensor secara presisi menggunakan operasi numerik standar (`f32`), yang menghasilkan nilai galat konstan sebesar 1.0°C untuk suhu dan 2.0% untuk kelembaban sesuai dengan asumsi *offset* pada sensor DHT11 virtual. Ekstrapolasi gas pada MQ135 juga berjalan baik dengan kalkulasi CO_2 sebesar 210.0 ppm dan NH_3 sebesar 420.0 ppm.

7.2 Skenario 2: Peringatan Ambang Batas (Alarm Aktif)

Pada pengujian kedua, fitur *Update Nilai* digunakan untuk memasukkan data aktual baru (suhu 2.0°C dan kelembaban 3.0%). Pada skenario ini, diasumsikan batas *Set Point* yang diatur sangat ketat (misalnya di bawah 3.0%). Seperti terlihat pada Gambar ??, ketika variabel `self.dht11_humid()` melampaui *Set Point*, kontrol sistem langsung dieksekusi secara *real-time*.

Peralihan Boolean menjadi *true* seketika memerintahkan aktuator untuk mengubah status menjadi **AKTIF**, ditandai dengan interupsi pesan peringatan “!! Humidity melebihi Set Point !!”. Pembacaan MQ135 turut menurun secara proporsional menjadi CO_2 sebesar 15.0 ppm dan NH_3 sebesar 30.0 ppm. Hal ini membuktikan bahwa enkapsulasi dan aliran logika program Rust yang dikembangkan telah responsif untuk diterapkan pada *plant* instrumentasi.

8 Analisis Hasil

Analisis menunjukkan bahwa sistem responsif terhadap perubahan data dinamis. Penggunaan tipe data `f32` memberikan presisi yang cukup untuk angka di belakang koma pada pembacaan suhu dan kadar gas ppm. Integrasi logika kontrol pada metode `aktuator_aktif()` berhasil menyatukan kondisi sensor digital dan kondisi aktual sesuai standar sistem IoT.

9 Kesimpulan

Aplikasi monitoring berbasis Rust ini berhasil memenuhi kriteria ETS, mencakup penggunaan OOP, validasi input, dan komputasi numerik. Sistem simulasi ini membuktikan bahwa Rust sangat efektif digunakan untuk logika kontrol instrumentasi cerdas.

References

- [1] R. Garg, A. Kumar, R. Dayana, and S. Singh, “Advanced Air Quality Monitoring System using IoT and Sensor Technology,” *2025 3rd International Conference on Intelligent Data Communication Technologies and Internet of Things (IDCIoT)*, Chennai, India, 2025, pp. 687-692.
- [2] S. Chanthakit and C. Rattanapoka, “MQTT Based Air Quality Monitoring System using NodeMCU and Node-RED,” *2018 Seventh ICT International Student Project Conference (ICT-ISPC)*, Bangkok, Thailand, 2018, pp. 1-5.